

Supply Chain Analytics Project — Working Log

Vijay Prakash

September 2026

About this document

This is a running log of the analysis process for the Supply Chain Analytics project — every business question, the SQL used to answer it, the Excel cross-check, and the reasoning behind each conclusion. It is being built question-by-question as the project progresses, and will form the basis of the final 1-page insight summary and resume write-up.

Tools used: MySQL (query layer) · Excel (pivot tables, charts, cross-validation) **Dataset:** DataCo Smart Supply Chain — 180,519 order-item records, 2015–2017

Q1 — Does a faster shipping mode actually deliver more reliably?

Why this question

Shipping Mode is one of the few fields directly under the company’s control (unlike, say, customer location). If certain modes are structurally unreliable, that’s an actionable finding — worth checking before looking at anything else.

SQL approach

Goal: For each shipping mode, find what percentage of orders were marked late.

Key idea used: `Late_delivery_risk` is a flag column containing only 0 or 1 (0 = on time, 1 = late). Averaging a column of 0s and 1s directly gives the proportion of 1s — e.g. if half the values are 1, the average is 0.5, i.e. 50%. This avoids having to manually count and divide.

```
SELECT `Shipping Mode`,
       COUNT(*) AS total_orders,
       ROUND(AVG(Late_delivery_risk) * 100, 1) AS late_pct
FROM supply_chain.orders
GROUP BY `Shipping Mode`
ORDER BY late_pct DESC;
```

SQL result

Shipping Mode	Total Orders	Late %
First Class	27,814	95.3%
Second Class	35,216	76.7%
Same Day	9,737	45.7%
Standard Class	107,752	38.1%

First interpretation (and the correction that mattered)

Initial read: “Standard Class delivers fastest and most reliably.”

Correction: `Late_delivery_risk` measures lateness *relative to the mode’s own promised delivery window*, not absolute shipping speed. So this doesn’t mean Standard Class is faster — it means Standard Class is more likely to meet whatever deadline it set for itself. First Class isn’t necessarily slower in real days; it may simply be promising something harder to hit.

Corrected finding: Standard Class carries ~60% of all order volume and misses its own delivery promise 38% of the time. First Class carries ~15% of volume and misses its promise 95% of the time.

Digging into the root cause

Question raised: Is First Class actually handled worse operationally, or does it simply promise an unrealistic delivery window?

Approach: Check how many distinct values exist for `Days for shipment (scheduled)` among First Class orders only.

```
SELECT COUNT(DISTINCT `Days for shipment (scheduled)`) AS distinct_schedule_values
FROM supply_chain.orders
WHERE `Shipping Mode` = 'First Class';
```

Result: 1 — every single First Class order is scheduled for a 1-day delivery window, with no exceptions.

Conclusion (root cause, validated)

First Class orders are scheduled for a 1-day delivery window 100% of the time, which is almost certainly why they show a 95.3% late-delivery rate — the promise itself is unrealistic, not necessarily a fulfillment failure.

Excel cross-validation

The same result was independently reproduced in Excel using two pivot tables on the same imported dataset (180,519 rows, confirmed via `Ctrl+End` and the Excel row-count indicator):

1. **Pivot 1** — Rows: `Shipping Mode`, Values: Average of `Late_delivery_risk` → produced the exact same 95.3% / 76.7% / 45.7% / 38.1% split as SQL.
2. **Pivot 2** — Filter: `Shipping Mode` = First Class, Rows: `Days for shipment (scheduled)` → returned a single row (value = 1), confirming the 100% finding independently of the SQL result.

Chart: Clustered column chart, First Class bar highlighted in red, y-axis formatted as a percentage, with a caption box stating the root-cause finding directly on the chart.

Recommendation (for final report)

The company should investigate whether the First Class shipping promise (1-day delivery) is operationally realistic given current fulfillment capacity — either by extending the promised window or by identifying what would need to change to reliably support a 1-day guarantee.

Q2 — Do top-revenue product categories also drive the most profit?

Why this question

Revenue and profit aren't the same thing — a category can sell a lot and still be a weak earner if its margin is thin. Before assuming “our biggest sellers are our biggest earners,” it's worth checking whether that's actually true.

SQL approach

Goal: For each category, get total revenue, total profit, and profit margin (profit as a % of revenue), sorted by revenue so the biggest sellers are visible first.

Key ideas used: - Every non-grouped column in a `GROUP BY` query must be wrapped in an aggregate function (`SUM`, `AVG`, etc.) — otherwise MySQL can't determine which row's value to show for a category with thousands of underlying rows. - Margin has to be calculated by dividing two `SUM()` expressions directly — a column alias created in the same `SELECT` can't always be reused within it, so the `SUM()` expressions are repeated inside the division.

```
SELECT `Category Name`,
       SUM(Sales) AS total_revenue,
       SUM(`Benefit per order`) AS total_profit,
       SUM(`Benefit per order`) / SUM(Sales) * 100 AS profit_margin_percentage
FROM supply_chain.orders
GROUP BY `Category Name`
ORDER BY total_revenue DESC;
```

SQL result (top categories by revenue)

Category Name	Total Revenue	Total Profit	Margin %
Fishing	6,929,653.69	756,220.77	10.91%
Cleats	4,431,942.78	494,636.92	11.16%
Camping & Hiking	4,118,425.57	427,455.57	10.38%
Cardio Equipment	3,694,843.20	383,011.10	10.37%
Women's Apparel	3,147,800.00	350,421.03	11.13%
Water Sports	3,113,844.68	325,146.96	10.44%
Men's Footwear	2,891,757.66	311,902.82	10.79%
Indoor/Outdoor Games	2,888,993.91	318,451.43	11.02%
Shop By Sport	1,309,522.04	129,813.96	9.91%

(Full result includes ~50 categories; smaller-revenue categories continue below this range.)

Interpretation

First check: Does the revenue ranking match the margin ranking? Answer: no — Fishing is #1 by revenue but sits at a fairly ordinary ~11% margin, in line with most other top-15 categories.

Second check: Looking further down the list (smaller-revenue categories), a few — Golf Bags & Carts, Soccer, Fitness Accessories — show noticeably higher margins (~15-17%), despite far lower revenue.

Anomaly spotted: Strength Training showed an unusually low 0.6% margin. Before treating this as meaningful, checked the sample size behind it:

```
SELECT COUNT(*) AS order_count, SUM(Sales) AS revenue
FROM supply_chain.orders
WHERE `Category Name` = 'Strength Training';
```

Result: 111 orders, ₹54,895.53 revenue — a large enough sample that this isn't just noise from a couple of unusual transactions, so it's a genuine anomaly worth flagging, even though Strength Training itself is a small category outside the top 15 by revenue.

Conclusion (validated)

Among the top 15 revenue-generating categories, profit margin stays remarkably flat (~10-13%), meaning revenue scale — not per-category efficiency — is what separates the biggest earners like Fishing from smaller ones. Outside the top 15, a few smaller categories (Golf Bags & Carts, Soccer, Fitness Accessories) show noticeably higher margins (~15-17%), suggesting a more efficient cost or pricing structure worth investigating for potential replication. Separately, Strength Training (111 orders, a meaningful sample size) showed an unusually low 0.6% margin — worth flagging as an anomaly for further investigation, though outside this chart's top-15 scope.

Note on rigor: the original draft of this conclusion suggested increasing top-category margins by “2-3%” as if that were an established fix. Revised after recognizing that the query only shows a margin *gap* exists — it doesn't establish *why*, or that the underlying cost/pricing structure behind smaller categories' higher margins could actually be replicated elsewhere. The final wording reflects that distinction (a gap worth investigating, not a proven solution).

Excel cross-validation

- Pivot table (new sheet, Q2_CategoryMargin): Rows = Category Name, Values = Sum of Sales and Sum of Benefit per order, plus a Calculated Field (Profit Margin % = Benefit per order / Sales × 100).
- Fishing's revenue (~6,929,653) matched the SQL result exactly, confirming the Excel import and pivot logic are correct.
- Sorted descending by Sum of Sales — same category order as SQL.

Chart: Combo chart (clustered columns for Total Revenue on the primary axis, line for Profit Margin % on the secondary axis), limited to the top 15 categories by revenue for readability. Title states the finding directly: “Profit Margin % is almost flat (~10-13%) among the top categories.” Both axes labeled (Total Revenue (₹) / Profit Margin (%)).

Recommendation (for final report)

Investigate what drives the higher margins in Golf Bags & Carts, Soccer, and Fitness Accessories (pricing, supplier cost, discounting patterns) to assess whether that structure is replicable in higher-revenue categories like Fishing — where even a small margin gain would have an outsized absolute profit impact given its revenue scale. Separately, investigate the Strength Training category's unusually low margin as a potential cost or pricing issue.

Q3 — Which regions combine high late-delivery risk with low profit margin?

Why this question

After finding a strong shipping-mode effect (Q1) and a category-driven margin effect (Q2), the natural next check is geography — is there a specific region that’s underperforming on *both* fronts at once, which would need a more localized fix?

SQL approach

Goal: For each region, calculate late-delivery rate (same `AVG()` trick as Q1) and profit margin (same ratio calculation as Q2), together in one query — so both metrics can be compared side by side per region.

Column correction made during setup: Initially considered using `Order Country`, but that’s too granular (dozens of small groups). The dataset has a separate, broader `Order Region` column that actually matches what “region” means in this business question.

```
SELECT `Order Region`,
       AVG(Late_delivery_risk) AS late_rate,
       SUM(`Benefit per order`) / SUM(Sales) * 100 AS profit_margin_percentage
FROM supply_chain.orders
GROUP BY `Order Region`
ORDER BY late_rate DESC;
```

SQL result (selected rows)

Order Region	Late Rate	Margin %
Southeast Asia (highest late rate group)	~55.5%	~10.9%
... (most regions)	53-58%	10-13%
Canada (lowest late rate)	48.8%	12.8%

(Full result spans ~23 regions; the full spread is 48.8% to 58% late rate, 9.1% to 13.5% margin.)

Interpretation

Unlike Q1 (a dramatic 38% vs 95% split) and Q2 (some categories showing 15-17% margins vs. a flat ~11% for top sellers), **regions show almost no meaningful spread** — late rates cluster in a narrow 48-58% band, and margins cluster in a 10-13% band. Canada stands out slightly as the lowest late-rate region (48.8%), but with an unremarkable margin (12.8%) — not a dramatic outlier on both fronts simultaneously.

Conclusion (validated, and connected to Q1 + Q2)

Late-delivery rates (48-58%) and profit margins (10-13%) are both fairly uniform across regions, with Canada showing the lowest late rate (48%) but no other region standing out sharply on either metric. This suggests region is not a major driver of either problem — consistent with Q1’s finding that shipping mode (specifically the 1-day scheduling window on First Class orders) is a far stronger predictor of lateness, and Q2’s finding that category, not geography, is what drives margin variation.

This is a legitimate finding in its own right: it tells the business *where the problem isn’t*, which narrows

attention toward the two levers that actually matter (shipping mode design, category-level margin structure) rather than a geographic rollout of fixes that wouldn't move the needle.

Excel cross-validation

- Pivot table (new sheet, Q3_Region): Rows = Order Region, Values = Average of Late_delivery_risk and a Calculated Field for Profit_Margin% (same formula as Q2).
- Canada's late rate matched SQL exactly (48.8%), confirming the cross-check.
- Learned to sort a pivot table by a *value* column correctly: right-click a data cell (not the header) → Sort → Largest to Smallest — the header dropdown only filters/sorts row labels, not the underlying values.

Chart: Plain (non-pivot) bar chart, built by copying the two relevant columns out as values first — necessary because a PivotChart stays linked to every field in the source pivot table, so removing one field from the chart would have deleted it from the table too. Chart deliberately has **no highlighted bar** (unlike Q1's red First Class bar) since the finding here is uniformity, not an outlier — and the y-axis is zoomed to 44-60% (not starting at 0%) so the genuine small variation is visible without exaggerating it into a false dramatic trend.

Recommendation (for final report)

No region-specific intervention is indicated by this data. Continued focus should stay on shipping-mode scheduling (Q1) and category-level margin structure (Q2), which are the two levers actually showing meaningful variation.

Q4 — How do order volume and profit trend over 2015-2017?

Why this question

Before drawing conclusions from a time-based trend, it's important to check whether the trend is real and complete across the whole date range — a partial or cut-off dataset can easily be mistaken for a genuine business pattern if not checked first.

SQL approach

Goal: Group orders by calendar month (not exact date/time) and get order count and total profit per month, in chronological order.

Key idea used: `DATE_FORMAT(order_date, '%Y-%m')` converts a full timestamp into just a “year-month” text value (e.g. “2015-06”), so that all orders placed anywhere within the same month collapse into one group when used with `GROUP BY`.

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS Order_month,
       COUNT(*) AS Number_of_orders,
       SUM(`Benefit per order`) AS profit
FROM Supply_chain.orders
GROUP BY order_month
ORDER BY order_month;
```

Anomaly discovered

The unfiltered result showed order volume holding steady at ~4,900-5,400 orders/month from Jan 2015 through Sep 2017, then dropping sharply to ~2,055-2,255 orders/month for the final four months (Oct 2017-Jan 2018).

Investigation — was this a partial data export?

```
SELECT MIN(order_date) AS earliest, MAX(order_date) AS latest
FROM supply_chain.orders;
```

Result: earliest = 2015-01-01 00:00:00, latest = 2018-01-31 23:38:00. The data runs to the very last minute of January 2018 — ruling out a simple “cut off mid-month” explanation.

Second check — is the drop proportional (real slowdown) or erratic (data quality issue)?

Calculated profit-per-order for the last five months by hand from the query results:

Month	Orders	Profit	Profit ÷ Order
2017-09	5,189	122,462.39	23.6
2017-10	2,255	113,447.17	50.3
2017-11	2,055	67,791.25	33.0
2017-12	2,124	65,837.63	31.0
2018-01	2,123	33,841.89	15.9

Profit-per-order is unstable across these months (swinging from 23.6 up to 50.3, then down to 15.9) rather than holding steady — inconsistent with a clean, proportional real-world slowdown.

Conclusion on the anomaly

Order volume holds steady at roughly 5,000-5,400/month from 2015 through September 2017, then drops sharply (to ~2,100-2,255/month) for the final four months of the dataset (Oct 2017-Jan 2018). Profit-per-order over this same period is unstable rather than holding steady as would be expected from a clean, proportional slowdown. Combined with the fact that the drop coincides exactly with the dataset’s final four months, this looks like a data completeness issue (the export likely tapering off near its cutoff date) rather than a genuine business decline, and is flagged as a limitation rather than treated as a real trend.

Decision: Excluded Oct 2017-Jan 2018 from the trend analysis. Original data left untouched; exclusion applied only via `WHERE` filter for this specific analysis, and stated openly rather than silently dropping rows.

Filtered SQL result

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS Order_month,
       COUNT(*) AS Number_of_orders,
       SUM(`Benefit per order`) AS profit
FROM Supply_chain.orders
WHERE order_date < '2017-10-01'
GROUP BY order_month
ORDER BY order_month;
```

Result: 33 clean months (Jan 2015-Sep 2017), order volume consistently in the ~4,900-5,400 range with no sudden breaks.

Final conclusion

Excluding the incomplete final four months, order volume is evenly distributed across the full Jan 2015-Sep 2017 period (~4,800-5,400 orders/month), with no meaningful growth or decline trend. This is a stable, mature order volume pattern rather than a business in flux.

Excel cross-validation

- Pivot table (new sheet, `Q4_MonthlyTrend`): Rows = `order_date`, grouped by both Month and Year (right-click a date cell in the pivot body → Group → select Months + Years together, to avoid merging Jan 2015 and Jan 2016 into one bucket).
- Applied a Date Filter (Before 10/1/2017) matching the SQL `WHERE` clause.
- Spot-check: Sep 2017 showed 5,189 orders and ₹122,462.39 profit — exact match to SQL.
- **Mistake caught and fixed:** first chart attempt copied the pivot table's auto-generated year subtotal rows (e.g. "2015: 62,650") along with the actual month rows, producing three false spikes in the line chart. Fixed by turning off subtotals entirely (PivotTable Design tab → Subtotals → Do Not Show Subtotals) before copying data for the chart.

Chart: Line chart, order volume by month, y-axis scaled to 4200-5600 (not starting at 0) to make the genuine month-to-month variation visible. Year boundaries (2015 | 2016 | 2017) added as a secondary label beneath the month axis for readability across the full 33-month span. Title states the finding directly: *"Order volume is evenly distributed across months from 2015-2017 (~4800-5400)."* A text box notes the excluded anomaly period and the reasoning for excluding it.

Recommendation (for final report)

Order volume is stable and mature — no seasonal growth/decline pattern requiring action. Flag the Oct 2017-Jan 2018 data gap as a data-quality limitation in the final report rather than a business finding, and note that any stakeholder relying on this dataset for recent-period decisions should be aware the last four months are unreliable.

Q5 — Does discounting correlate with higher sales volume or profit ratio?

Why this question

Discounting is a lever the company actively controls, and it's often assumed to drive volume. Before accepting that assumption, it's worth checking whether the data actually supports it — and what discounting costs in margin if it doesn't.

SQL approach

Goal: Group orders into discount-rate bands (since the raw rate is a continuous number, grouping by the exact value would create too many tiny, noisy groups) and compare average sale size and profit margin across bands.

Key idea used: A `CASE WHEN` block sorts each row into a labeled bucket before grouping — the entire `CASE...END` expression has to sit directly inside the `SELECT` list as the column being grouped by, not be referenced before it's defined.

```
SELECT
```

```

CASE
  WHEN `Order Item Discount Rate` = 0 THEN '0%'
  WHEN `Order Item Discount Rate` < 0.1 THEN '0-10%'
  WHEN `Order Item Discount Rate` < 0.2 THEN '10-20%'
  ELSE '20%+'
END AS discount_bucket,
AVG(Sales) AS Avg_Sales,
SUM(`Benefit per order`) / SUM(Sales) * 100 AS profit_margin_percentage
FROM supply_chain.orders
GROUP BY discount_bucket
ORDER BY discount_bucket;

```

SQL result

Discount Bucket	Avg Sales	Margin %
0%	203.67	13.09%
0-10%	203.74	11.48%
10-20%	203.80	10.16%
20%+	203.84	9.03%

Interpretation

Average Sales is essentially flat (203.67 → 203.84) across every discount tier — discounting shows no evidence of increasing how much customers spend per order. **Profit margin declines steadily and consistently** as discount rate increases, from 13.09% down to 9.03%.

Correlation vs. causation check

Concern raised: the aggregate result could be misleading if certain naturally low-margin categories simply happen to be discounted more often — in which case category, not discounting itself, would be the real driver.

Test 1 — isolate a single category (Fishing, the highest-revenue category from Q2):

```
... (same query structure) WHERE `category name` = 'fishing'
```

Result: 13.76% → 11.55% → 10.32% → 9.01% — nearly identical decline pattern to the aggregate result, suggesting the trend is not merely a category-mix artifact.

Test 2 — compare multiple categories side by side (2 high-revenue: Fishing, Cleats; 3 high-margin/low-revenue outliers from Q2: Golf Bags & Carts, Soccer, Fitness Accessories), grouping by both category and bucket:

```

SELECT `category name`,
CASE ... END AS discount_bucket,
AVG(Sales) AS Avg_Sales,
SUM(`Benefit per order`) / SUM(Sales) * 100 AS profit_margin_percentage
FROM supply_chain.orders
WHERE `Category Name` IN ('Fishing', 'Cleats', 'Golf Bags & Carts', 'Soccer', 'Fitness Accessories')
GROUP BY `Category Name`, discount_bucket
ORDER BY discount_bucket, `Category Name`;

```

Result: The clean decline held for Fishing and roughly for Cleats (both high-volume categories). For the three smaller categories (Fitness Accessories, Golf Bags & Carts, Soccer), margins swung erratically bucket to bucket instead of declining steadily (e.g. Soccer ranged from 25.6% down to 0.96% with no consistent direction).

Conclusion (validated across two levels of granularity)

Discounting shows no evidence of increasing order value — average sales stay flat (~₹203-204) across all discount tiers. Profit margin declines steadily and consistently as discount rate increases, from ~13% (no discount) down to ~9% (20%+ discount). This pattern holds cleanly for high-volume categories (Fishing, Cleats) when checked in isolation, which confirms it's a real relationship and not simply a category-mix artifact in the aggregate data. For lower-volume categories, the same bucketing produces erratic, non-monotonic results — most likely because a discount bucket within a smaller category contains too few orders for a stable average, rather than evidence that discounting behaves differently in nature there. Any category-specific action should first confirm sufficient order volume per bucket before trusting the trend.

Excel cross-validation

- Pivot table (new sheet, `Q5_Discount`): Rows = `Order Item Discount Rate` rounded to 2 decimals (~18 distinct rates), Values = Average of `Sales` and a Calculated Field for margin.
- 0% discount row showed 13.09% margin — exact match to SQL's "0%" bucket.
- Also tried grouping the raw rate into 3 broad bands (0-0.1, 0.1-0.2, 0.2-0.3) using the pivot table's numeric Group feature — a valid alternative to SQL's `CASE WHEN`, though it folds the "exactly 0%" row into the first band, so it doesn't match the SQL buckets exactly. Ultimately used the finer, ~18-point rounded version instead, since more data points make the steady decline more convincing than 3-4 buckets do.

Chart: Scatter plot (line + markers) of Margin % against Discount Rate, using the granular ~18-point data rather than the coarser 3-4 bucket version, since the finer granularity shows the decline is consistent across many points rather than an artifact of a few arbitrary groupings.

Recommendation (for final report)

Discounting does not appear to drive higher order value and comes at a measurable, consistent margin cost — validated across two categories in isolation. A blanket increase in discounting is not supported by this data. However, before recommending any specific discount policy change, this relationship should be checked within each category individually (with sufficient order volume per bucket) rather than applied as a single company-wide rule, since category-level behavior was shown to vary.

(All five business questions completed and cross-validated in SQL and Excel. Section below documents the Power BI dashboard build — the third tool used to independently re-verify every finding, plus a full interactive dashboard for presentation.)

Power BI Dashboard

Why a third tool

SQL and Excel had already answered and cross-validated all five business questions. Power BI was added to (a) verify the same findings a third time in a genuinely different engine, and (b) produce one polished, interactive artifact that presents all five findings together, rather than five separate spreadsheets — the piece most useful for showing to a non-technical audience.

Data connection

Started with the simple path: imported `supply_chain_cleaned.csv` directly (**Get Data → Text/CSV**), same file used in Excel. A more advanced path — connecting Power BI directly to the live MySQL database (`Get Data → MySQL Database`) — was identified as a follow-up upgrade, to complete the full pipeline story (MySQL → Power BI with no CSV step in between), but not done yet as of this log.

Visual 1 — Shipping mode lateness (Q1)

Built a Clustered Column Chart: `Shipping Mode` on X-axis, `Late_delivery_risk` on Y-axis.

Bug hit and fixed: the field defaulted to **Sum** aggregation rather than Average — same trap as Excel's default pivot behavior. Fixed via the Y-axis field's aggregation dropdown (Sum → Average). Result matched SQL/Excel exactly (95.32%, 76.63%, 45.74%, 38.07%).

Formatting notes: - Percentage display isn't set from the chart's axis panel (where it might be expected) — it's set at the **field level**: select the column in the Data pane → **Column tools** ribbon tab → Format → Percentage. This formats the field everywhere it's used, including the chart. - Highlighted only the First Class bar in red using **conditional formatting** on the column color (Format visual → Columns → Colors → fx → rule: if `Shipping Mode` is "First Class" → red). Only one rule was needed — any category not matched by a rule keeps the default color automatically.

Visual 2 — Category revenue vs. margin (Q2)

Required a new **DAX measure**, since Power BI charts can't divide two raw fields directly the way Excel's Calculated Field or SQL's `SUM()/SUM()` can:

```
Profit Margin % = DIVIDE(SUM(supply_chain_cleaned[Benefit per order]),  
SUM(supply_chain_cleaned[Sales]))
```

`DIVIDE()` is Power BI's safe-division function (avoids errors on a zero denominator). The measure is left as a raw ratio (no $\times 100$) — the $\times 100$ and "%" symbol are applied purely as a *display format* on the measure (Format → Percentage), not baked into the formula. Multiplying by 100 manually in the formula *and* applying a percentage format would have double-counted (0.109 → 1090%).

Bug hit and fixed: first attempt to build the combo chart threw:

```
Calculation error in measure 'supply_chain_cleaned'[Profit Margin %]:  
The function SUM cannot work with values of type String.
```

Diagnosed by checking the Σ icon next to each field in the Data pane — `Benefit per order` showed Σ (numeric), `Sales` did not (text). Fixed in Power Query (**Transform data** → click the type icon on the `Sales` column → Decimal Number → Replace Current Step → Close & Apply). This is the same category of bug hit twice before (Excel's `Sales` column, MySQL's `Customer Zipcode` column) — CSV imports silently misjudging a numeric column as text based on a small sample of early rows.

Built as a **Line and clustered column chart**: `Category Name` on X-axis, `Sales` (Sum) as columns, `Profit Margin %` measure as the line. Filtered to **Top 15 by Sales** (Filters pane → Category Name filter → change type to Top N → N=15 → By value: Sales) — same reasoning as the Excel version: ~50 categories on one axis is unreadable, and more data points support a "flat trend" claim more convincingly than a coarse 3-4 bucket summary.

Visual 3 — Region uniformity (Q3)

Clustered Bar Chart: `Order Region` on X-axis (Y-axis in a horizontal bar), `Late_delivery_risk` (Average).

Mistake caught and fixed: first build accidentally left `Profit Margin %` in the axis well instead of `Late_delivery_risk` — caught because the chart's *title* ("% Late Delivery risk is uniform...") didn't match

what the *axis* was actually plotting (5%/10%/15% range, not 40-60%). General lesson: always check the title against the actual axis values, not just whichever one looks reasonable on its own.

Sorted using the chart's ... **menu** → **Sort by** the value field, matching Excel's Canada-lowest ordering.

Visual 4 — Monthly trend (Q4)

Line Chart: `order_date` on X-axis, Count of an order-item field on Y-axis.

Bug hit and fixed (twice): 1. First attempt showed an unnaturally smooth curve across only 4 points (one per year) — caused by the date field still being aggregated at the **Year** level while “Month” also sat in the same field well, so Power BI interpolated between just 4 points rather than plotting ~37 real monthly values. Fixed by drilling down to the Month level on the chart itself (right-click chart → Drill Down, or the expand-level control in the chart's corner) so all ~37 months render individually. 2. The exclusion filter (`order_date < 10/01/2017`) showed a warning triangle — the typed date string was ambiguous (day-first vs. month-first). Fixed by using the filter's calendar picker instead of typing the date as text, removing the ambiguity.

Y-axis manually zoomed to 4200-5600 (matching Excel) so the real month-to-month variation is visible. Added a text box noting the excluded Oct 2017-Jan 2018 anomaly and referencing the fuller investigation in this log.

Visual 5 — Discount vs. margin (Q5)

Required a **calculated column** (not a measure) to bucket the continuous discount rate, since bucket assignment needs to happen per-row, before aggregation:

```
Discount Bucket =  
IF('supply_chain_cleaned'[Order Item Discount Rate] = 0, "0%",  
IF('supply_chain_cleaned'[Order Item Discount Rate] < 0.1, "0-10%",  
IF('supply_chain_cleaned'[Order Item Discount Rate] < 0.2, "10-20%", "20%+")))
```

Built as a Line and clustered column chart: `Discount Bucket` on X-axis, `Sales` (Average, not Sum — matching the “average order value” question) as columns, `Profit Margin %` measure as the line. Result matched SQL/Excel: flat average sale (~₹200-210) across buckets, margin declining from ~13% down to ~9%.

Dashboard assembly

KPI cards (4): Total Orders (Count Distinct of Order Id — deliberately distinct from the SQL/Excel “180,519 order-items” figure, since an order can contain multiple line items; landed on 65,752 distinct orders, ~2.7 items/ order on average), Avg. Late Delivery Risk (Average, ~54.8%), Overall Profit Margin (the `Profit Margin %` measure directly, ~11%), Total Revenue (Sum of Sales, ~₹36.78M).

Layout: title text box across the top; 4 KPI cards in a row beneath it; the shipping-mode chart (Q1) enlarged as the dashboard's “hero” visual since it's the strongest, most specific finding; the monthly trend (Q4) beside it; category (Q2), region (Q3), and discount (Q5) charts in a smaller row below as supporting evidence. Reasoning: KPIs orient the viewer in five seconds; one hero visual gives the dashboard a clear focal point rather than five equally-weighted charts competing for attention.

Polish applied: gridlines/snap-to-grid turned on before any manual resizing; a light gray canvas background with white card backgrounds behind every visual and KPI (so elements read as distinct cards rather than blending into the page); visible gaps between every element (nothing flush against its neighbor); a colored header band behind the title; selected-all-then-Align/Distribute used to evenly space the KPI row and the bottom chart row rather than manual pixel-nudging.

Cross-validation summary

All five Power BI visuals independently reproduced the same headline numbers already validated in SQL and Excel (95.32%/76.63%/45.74%/38.07% for Q1; ~11% flat margin for Q2; 48-58% uniform range for Q3; the same monthly pattern with the same excluded anomaly for Q4; the same 13%→9% margin decline for Q5) — three independent tools, same conclusions, for all five questions.

Still open

- Reconnect the dashboard to the live MySQL database directly (currently CSV-based) to complete the full SQL → Power BI pipeline without a manual export step in between.
 - Export the finished dashboard as PDF/image and the Excel workbook for inclusion in a GitHub repository alongside the SQL and log files.
-

(Project complete pending the MySQL-direct-connection upgrade and GitHub packaging. Ready for the final 1-page insight summary and resume write-up.)